

BellyUp Buffet Simulation

Redesign Implementation Plan

AnyLogic Desktop Simulation — Full Logic & Flow Reference

1. Table Assignment Strategy

The hybrid packing strategy determines how groups of guests are seated to minimize wasted seats and avoid fragmented seating across multiple tables.

1.1 Original Strategy (Baseline)

Group Size	Assignment
1 – 4	One 4-seater table
5 – 7	Two 4-seater tables
8+	One 10-seater table

Problem with original strategy: A group of 12 receives a 10-seater + 2 extra seats at another 10-seater, wasting 8 seats and splitting the group.

1.2 Recommended Strategy: Hybrid Packing

Group Size	Assignment	Max Capacity	Max Empty Seats
1 – 4	1 × 4-seater table	4	3
5 – 8	1 × 10-seater table	10	5
9 – 12	1 × 10-seater + 1 × 4-seater	14	5
13 – 16	2 × 10-seater tables	20	7
17 – 20	2 × 10-seater + 1 × 4-seater	24	7
21+	Continue pattern (add 10-seater per 10 guests)	—	≤ 4

1.3 Why Hybrid Packing is Better

- Groups of 5–8 sit at a single 10-seater instead of two separate 4-seaters, avoiding fragmented families.
- Overflow above a 10-seater fills a 4-seater supplement rather than burning a full second 10-seater.
- Maximum chair waste per group: 4 empty seats — predictable and acceptable for a buffet context.
- Real-world accuracy: buffets seat smaller groups at larger round tables naturally.

1.4 General Formula (Java)

```
int num10Seaters = N / 10;           // whole 10-seater blocks
int remainder    = N % 10;

// remainder 1-4 → add one 4-seater
// remainder 5-10 → add one more 10-seater (or tolerate 5 empty seats)
```


2. Currently Existing Flow (Mapped from .alp)

The following flow represents the blocks currently present in the AnyLogic model, mapped from the .alp file.

2.1 Guest Flow

Block	Type	Description
pedSource1	PedSource	Generates pedestrian groups
walkToReception	PedService	Group queues and is served at reception
pedSelectOutput	PedSelectOutput	Routes group: out1 = go to table, out5 = early exit (unused)
pedGoTo1	PedGoTo	Walk to claimed table node
seize4	PedService/Seize	Claim the table node
goToAssignedTable	PedGoTo	Navigate to the physical table
secureAssignedTable	PedWait	Seat the group at the table
pedGroupDisassemble	PedGroupDisassemble	Dissolve group; each member retains groupId
goBackToTable	PedGoTo	Each member walks to their assigned seat attractor
foodSelectionWaiting	PedWait	Member sits at seat
goToServingStation	PedGoTo	Walk to a randomly chosen food station
pedSelectOutput1	PedSelectOutput	out1 = eating loop; out5 = go to cashier
eatingTime	PedWait	Eat at station or seat
pedGroupDisassemble1	PedGroupDisassemble	Dissolve for cashier path
goToCashier	PedGoTo	Walk to cashier
pedSink1	PedSink	Guest exits simulation

2.2 Chef / Waiter Refill Chain (Existing)

```
triggerRefill (Source)
  → grabChef (Seize)
  → cookingTime (Delay)
  → freeChef (Release)
  → grabWaiter (Seize)
  → refillTray (Delay)
  → walkToBuffet (MoveTo)
  → freeWaiter (Release)
  → sink
```

3. Target Logic — Desired Flow (Per Design Brief)

The following phases describe the complete redesigned simulation logic. All phases are sequential per guest group unless otherwise noted.

Phase 1: Group Generation & Reception

- PedSource generates groups; each member is assigned an auto-incremented groupId.
- The groupId is shared across all members of the same group.
- Group walks to Reception → PedService assigns table node(s) using the Hybrid Packing strategy.
- On onExit of the service block: each member receives assignedTable and assignedSeatIndex.

Phase 2: Seating

- Group navigates to assigned table node via PedGoTo.
- PedGroupDisassemble: dissolve group; retain groupId on each member.
- Each member walks to their specific attractor (seat index) via PedGoTo → PedWait.

Phase 3: Food Station Loop (Randomized)

- Each individual performs a random station selection via PedSelectOutput (equal or weighted probability):
 - Seafood Area
 - Dimsum Area
 - Buffet Table (main hot dishes)
 - Beverages Station
- Member walks to chosen station and reduces the station's food variable (e.g., seafoodLevel -= 1).
- Refill trigger check: if stationFoodLevel <= 20% of maxLevel, trigger the chef/waiter refill chain for that specific station.
- Member returns to assigned seat via PedGoTo to assignedTable.getAttractor(assignedSeatIndex).

Phase 4: Post-Meal Decision Loop

- At seat: PedSelectOutput probability gate evaluates whether the member gets more food or is done eating.
 - Get more food (60%): loop back to food station selection.
 - Done eating (40%): set isDoneEating = true; proceed to Group Readiness Check.

Phase 5: Group Readiness Check

Before any payment occurs, all group members must be done eating. This gate enforces group synchronization.

- Each member who decides to stop eating navigates back to the assigned table and enters a PedWait assembly hold point.
- On each member's arrival, the shared counter is incremented:

```
groupAssemblyCounter.put(agent.groupId,
    groupAssemblyCounter.get(agent.groupId) + 1);
```

- Check whether the full group has arrived:

```
if (groupAssemblyCounter.get(agent.groupId) == agent.groupSize) {  
    // All members done – designate paying member  
    agent.isPayingMember = true;  
}
```

- Members who finish eating earlier remain in PedWait at the table.
- Key rule: No member proceeds to the cashier until the full group is confirmed done eating.

Phase 6: Payment & Exit

- Once Phase 5 confirms all members are done, the paying member (isPayingMember = true) walks to the Cashier via PedGoTo.
- Payment is processed at the cashier node (PedService or PedWait delay).
- Paying member returns to the table via PedGoTo.
- Arrival of the paying member triggers PedGroupAssemble, reuniting all waiting members.
- The reassembled group exits together through PedSink.

4. Required Data Changes

4.1 Customer Agent Variables

Variable	Type	Purpose
groupId	int	Shared identifier across all group members
groupSize	int	Total member count in the group (set at spawn time)
assignedTable	RectangularNode	The physical table node assigned to this group
assignedSeatIndex	int	Which attractor (seat) the member occupies at their table
isPayingMember	boolean	True for the one member who handles payment
isDoneEating	boolean	True when member has exited the food selection loop

4.2 Main Model Variables

Variable	Type	Purpose
nextGroupId	int	Auto-increment counter; assigned in PedSource.onExit
seafoodFoodLevel	double	Current food level at seafood station
dimsumFoodLevel	double	Current food level at dimsum station
buffetFoodLevel	double	Current food level at main buffet table
beverageFoodLevel	double	Current food level at beverages station
seafoodFoodMax	int	Max capacity (e.g., 100) for seafood station
dimsumFoodMax	int	Max capacity for dimsum station
buffetFoodMax	int	Max capacity for main buffet station
beverageFoodMax	int	Max capacity for beverages station
groupAssemblyCounter	HashMap<Integer,Integer>	Tracks how many members per groupId have reached the assembly PedWait
availableTenSeaters	ArrayList<RectangularNode>	Pool of available 10-seater table nodes for assignment

4.3 Flow Blocks to Add or Modify

Block	Type	Change Required
pedSource1 onExit	Code	Assign ped.groupId = nextGroupId; nextGroupId++; set

Block	Type	Change Required
		ped.groupSize
walkToReception	PedService	onExit: call hybrid packing table assignment logic
goToServingStation	PedGoTo	Set destination to randomly chosen station node
Station visit delay	PedWait (new)	Short wait to pick up food; reduce food variable on exit
Refill trigger	Source + condition (new)	Trigger per station when food level <= 20%
foodDecisionGate	PedSelectOutput	Probability gate: 60% loop back, 40% done eating
groupReadinessWait	PedWait (new)	Hold members who are done eating until all are ready
assembleGroup	PedGroupAssemble (new)	Reassemble full group for synchronized exit
pedSink	PedSink	Final exit — triggered only after payment and reassembly

5. Refill Chain — Per Station

The current refill chain is global (one shared food_level variable). It must be extended to support all four stations independently.

5.1 Recommended Approach: Single Shared Chain with Station Variable

Use one chain with a refillTargetStation variable on Main. The waiter reads this variable to walk to the correct station and reset the correct food level.

```
// Trigger logic (per station, e.g., seafood):
if (seafoodFoodLevel <= 0.20 * seafoodFoodMax) {
    refillTargetStation = seafoodNode;
    refillTargetLevel   = "seafood";
    inject(triggerRefill, 1);
}

// On waiter arrival at station:
if (refillTargetLevel.equals("seafood")) seafoodFoodLevel = seafoodFoodMax;
if (refillTargetLevel.equals("dimsum"))  dimsumFoodLevel  = dimsumFoodMax;
if (refillTargetLevel.equals("buffet"))  buffetFoodLevel  = buffetFoodMax;
if (refillTargetLevel.equals("beverage")) beverageFoodLevel = beverageFoodMax;
```

Note: The Seize block for the chef handles queuing if multiple stations trigger refills simultaneously. The chain processes them in order.

6. Verification Plan

Since this is an AnyLogic desktop simulation (no automated test runner), all verification is performed manually via the 2D view, variable watch panel, and console output.

Test	Setup	Pass Criteria
Test 1 — Group ID Assignment	Run simulation; observe groupId label on each pedestrian.	All members of a group share the same groupId. Different groups have different IDs.
Test 2 — Hybrid Packing Table Assignment	Set PedSource to generate groups of sizes 4, 6, 8, 10, 12 (test each individually).	Groups 1–4: 4-seater. Groups 5–8: 10-seater. Groups 9–12: 10-seater + 4-seater.
Test 3 — Randomized Station Selection	Run simulation for several minutes; observe paths in the 2D view.	Customers visit different stations across runs. No single station is monopolized.
Test 4 — Food Level Reduction & Refill	Set station max = 10 (small value for fast testing); watch food level variables in real time.	Level decreases as customers visit. At $\leq 20\%$, chef starts cooking; waiter walks to correct station; level resets to 100%.
Test 5 — Return to Assigned Seat	Observe each customer after visiting a station.	Each customer returns to their own specific seat. Attractor index matches assignedSeatIndex.
Test 6 — Full Group Done Before Payment	Set group size to 5; artificially slow down 1–2 members' food loop so they finish later.	Fast members wait visibly in PedWait at table. Cashier trip only begins after the last member sets isDoneEating = true and the counter equals groupSize.
Test 7 — Synchronized Group Exit	Observe any group proceeding to PedSink.	All group members (same groupId) exit together as a reassembled group. No member exits before the paying member returns from cashier.